LibraryMS

Full Stack Library Management System

Django REST Framework • React + Vite • PostgreSQL

Version 1.0	Django 5.x	React 18	PostgreSQL

1. Project Overview

LibraryMS is a full-stack library management system built with Django REST Framework on the backend and React + Vite on the frontend. It supports three user roles: Admin, Librarian, and Member, each with their own dedicated portal and feature set.

> *LibraryMS is designed for real libraries — members self-register online, librarians manage day-to-day operations, and admins control the entire system.*

2. Tech Stack

Backend

Technology	Version	Purpose
Django	`5.x`	Web framework
Django REST Framework	`3.x`	REST API layer
Simple JWT	`5.x`	Staff authentication (JWT)

Technology	Version	Purpose
PyJWT	`2.x`	Member authentication (custom JWT)
PostgreSQL	`15+`	Primary database
django-cors-headers	`Latest`	Cross-origin resource sharing
python-dotenv	`Latest`	Environment variable management

Frontend

Technology	Version	Purpose
React	`18.x`	UI framework
Vite	`5.x`	Build tool & dev server
React Router	`6.x`	Client-side routing
Axios	`1.x`	HTTP client with interceptors
CSS Modules	—	Component-scoped styling

3. Features

Admin Portal

- Full dashboard with system-wide statistics
- Manage all books, members, borrowings, fines, and reservations
- Promote members to Librarian role
- View staff list and manage staff accounts
- Activity logs and reports
- Created exclusively via CLI command — cannot self-register

Librarian Portal

- Issue books to members with dropdown member/book selection
- Process book returns with automatic fine calculation
- Add, edit, and delete books with cover image support
- Register and manage library members
- View and manage reservations and fines

Member Portal

- Self-register online at /register
- Dashboard with notifications (overdue, due soon, unpaid fines)
- Browse and search books by title, author, or genre
- Reserve available books online
- View borrowing history and renew active loans (+14 days)
- View fines with amounts and payment status
- Manage profile (name, phone, address, password)

Borrowing & Fine System

- Due date set by librarian at time of issue (default 14 days)
- Automatic overdue detection — status changes to 'overdue' past due date
- Fine auto-created on return: days overdue × $0.50 per day
- Members can renew if not overdue (extends due date by 14 days)
- Overdue members must return and pay fine before renewing

4. Security Model

LibraryMS uses a three-tier role-based access control system with separate authentication flows for staff and members.

Role	How Created	Auth Method	Access Level
Admin	`CLI: python manage.py create_admin`	Django SimpleJWT	Full system access
Librarian	`Admin promotes a Member`	Django SimpleJWT	Books, members, borrowings, fines
Member	`Self-register at /register`	Custom PyJWT	Own data only

> *Important: Admin accounts cannot be created through the UI. Use python manage.py create_admin on the server. This prevents unauthorized admin account creation.*

5. Setup & Installation

Prerequisites

- Python 3.10+

- Node.js 18+
- PostgreSQL 14+
- Git

Backend Setup

Step 1 — Clone the repository

```
git clone https://github.com/yourname/library-system.git
cd library-system/library-backend
```

Step 2 — Create virtual environment

```
python -m venv venv
# Windows:
venv\Scripts\activate
# Mac/Linux:
source venv/bin/activate
```

Step 3 — Install dependencies

```
pip install django djangorestframework djangorestframework-simplejwt
pip install django-cors-headers psycopg2-binary python-dotenv PyJWT
```

Step 4 — Configure environment

Create a .env file in library-backend/:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
DB_NAME=library_db
DB_USER=postgres
DB_PASSWORD=yourpassword
DB_HOST=localhost
DB_PORT=5432
FINE_RATE_PER_DAY=0.50
```

Step 5 — Create database & run migrations

```
createdb library_db
python manage.py migrate
```

Step 6 — Create admin account

```
python manage.py create_admin
```

Follow the prompts to set email, name, and password.

Step 7 — Start backend server

```
python manage.py runserver
```

Backend runs on http://localhost:8000

Frontend Setup

Step 1 — Install dependencies

```
cd library-system
npm install
```

Step 2 — Configure environment

Create a .env file in library-system/:

```
VITE_API_URL=http://localhost:8000/api
```

Step 3 — Add background image

Download a library photo from unsplash.com/s/photos/library, rename it library-bg.jpg, and place it in library-system/public/library-bg.jpg

Step 4 — Start frontend

```
npm run dev
```

Frontend runs on http://localhost:5173

> *Both servers must be running simultaneously. Open two terminal windows.*

6. How to Use the App

Logging In

All roles use the same login page at /login. The system automatically detects your role and redirects you to the correct dashboard.

Role	Login With	Redirected To
Admin	Email set during create_admin	`/dashboard (Admin view)`

Role	Login With	Redirected To
Librarian	Email + changeme123 (first login)	`/dashboard (Librarian view)`
Member	Email + password from /register	`/member/dashboard`

Issuing a Book (Librarian Flow)

1. Member arrives at the library and requests a book
2. Librarian logs in and goes to Borrowings page
3. Click '+ Issue Book' — select member and book from dropdowns
4. Set due date (defaults to 14 days) and submit
5. Member can now see the borrowing in My Borrowings

Promoting a Member to Librarian

6. Login as Admin
7. Go to Admin Dashboard → Promote Members tab
8. Click 'Promote to Librarian' next to the member
9. Staff account is created — member logs in with their existing password

7. API Documentation

Base URL: http://localhost:8000/api/

Authentication Endpoints

Method	Endpoint	Auth	Description
POST	`/auth/login/`	Public	Staff login — returns access + refresh tokens
POST	`/auth/register/`	Public	Register new librarian account
POST	`/auth/token/refresh/`	Public	Refresh expired staff access token
POST	`/member/auth/register/`	Public	Member self-registration
POST	`/member/auth/login/`	Public	Member login — returns custom JWT
GET	`/member/auth/profile/`	Member	Get current member profile
PATCH	`/member/auth/profile/`	Member	Update member profile

Books Endpoints

Method	Endpoint	Auth	Description
GET	/books/	Staff	List all books with search
POST	/books/	Staff	Add a new book
PATCH	/books/:id/	Staff	Update book details
DELETE	/books/:id/	Staff	Delete a book
GET	/member/books/	Member	Browse books (search + genre filter)

Borrowings Endpoints

Method	Endpoint	Auth	Description
GET	/borrowings/	Staff	List all borrowings
POST	/borrowings/	Staff	Issue a book to a member
PATCH	/borrowings/:id/return/	Staff	Return a book + auto-create fine
GET	/member/borrowings/	Member	Get own borrowing history
PATCH	/member/borrowings/:id/renew/	Member	Renew a book (+14 days)

Fines Endpoints

Method	Endpoint	Auth	Description
GET	/fines/	Staff	List all fines
PATCH	/fines/:id/pay/	Staff	Mark fine as paid
GET	/member/fines/	Member	Get own fines

Reservations Endpoints

Method	Endpoint	Auth	Description
GET	/reservations/	Staff	List all reservations
PATCH	/reservations/:id/cancel/	Staff	Cancel a reservation
GET	/member/reservations/	Member	Get own reservations
POST	/member/reservations/	Member	Reserve a book
DELETE	/member/reservations/:id/	Member	Cancel own reservation

Admin Endpoints

Method	Endpoint	Auth	Description
GET	/admin/staff/	Admin	List all staff accounts
PATCH	/admin/staff/:id/	Admin	Update staff account
POST	/admin/promote/	Admin	Promote member to librarian
GET	/admin/logs/	Admin	View activity logs
GET	/admin/reports/	Admin	View system reports
GET	/dashboard/stats/	Staff	Get dashboard statistics

8. Screenshots

> *Add your own screenshots to this section. Suggested screenshots:*

- Login page — /login
- Admin Dashboard — /dashboard
- Books management page — /books
- Issue Book modal — /borrowings
- Member Dashboard — /member/dashboard
- Browse Books page — /member/books
- My Borrowings page — /member/borrowings

To add screenshots: In Word, go to Insert → Pictures → This Device and select your screenshot files.

9. Project Structure

Backend (library-backend/)

```
library-backend/
├── core/                  # Settings, URLs, auth
│   ├── settings.py
│   ├── urls.py
│   ├── member_urls.py     # Member portal routes
│   └── authentication.py  # OptionalJWTAuthentication
├── users/                 # Staff user model + auth
├── books/                 # Book model + views
├── members/               # Member model + auth views
├── borrowings/            # Borrowing model + views
├── fines/                 # Fine model + views
```

```
├── reservations/          # Reservation model + views
└── admin_panel/           # Admin-only views
```

Frontend (library-system/src/)

```
src/
├── pages/
│   ├── Login.jsx          # Unified login for all roles
│   ├── Register.jsx       # Member self-registration
│   ├── Books.jsx
│   ├── Members.jsx
│   ├── Borrowings.jsx
│   ├── Fines.jsx
│   ├── Reservations.jsx
│   ├── AdminDashboard.jsx
│   └── member/            # Member portal pages
│       ├── MemberDashboard.jsx
│       ├── BrowseBooks.jsx
│       ├── MyBorrowings.jsx
│       ├── MyFines.jsx
│       ├── MyReservations.jsx
│       └── MyProfile.jsx
├── components/            # Shared components
├── api.js                 # All API calls + interceptors
└── App.jsx                # Routing + role guards
```